UAV v3.4 Production 배포 가이드

제작일: 2026년 2월 25일 (수) 12:30 KST
기반: v3.4 4단계 통합 완성
목표: Phase 1-4 실행 → 통합 검증 → Production 배포

Executive Summary

v3.4 설계를 Production 환경으로 배포하기 위한 완전한 실행 가이드입니다. Phase 1-4 순차 실행 체크리스트, 통합 검증 절차, Docker 기반 Production 배포 전략을 포함합니다[1].

배포 목표

- Phase 1-4 순차 실행 (백엔드 → 프론트 → E2E → 검증)
- Definition of Done 전체 체크리스트 완료
- Docker Compose 기반 Production 환경 구축
- HTTPS/WSS 보안 통신 적용
- 모니터링 및 로깅 시스템 구축

---

1. Phase 1: 백엔드 통합 (30분)

1.1 파일 배치

sse_token.py 추가:

파일 위치: backend/app/sse_token.py

uav_v33_final/backend/app/sse_token.py → 프로젝트 backend/app/

cp uav_v33_final/backend/app/sse_token.py backend/app/

main.py 라우터 등록:

backend/app/main.py

from app.sse_token import router as sse_token_router

app.include_router(sse_token_router)

1.2 transport_sse.py 수정

기존 코드 수정 지점:

backend/app/transport_sse.py

from .sse_token import verify_sse_token

@router.get("/telemetry/stream")
async def telemetry_stream(
request: Request,
session_id: str = Query(...),
sse_token: str = Query(...), # token → sse_token 변경
last_event_id: str | None = Header(default=None, alias="Last-Event-ID"),
):
# sse_token 검증 (60초 TTL)
try:
user = verify_sse_token(sse_token)
except HTTPException as e:
raise e

```
# 이하 기존 로직...
```

1.3 keepalive 패치 적용

event_gen() 함수 수정:

backend/app/transport_sse.py

async def event_gen():
last_keepalive = time.time()

```
while not await request.is_disconnected():
    # 1. 텔레메트리 전송
    if queue.qsize() > 0:
        item = queue.get()
        yield f"data: {json.dumps(item)}\n\n"
        last_keepalive = time.time()

    # 2. keepalive (30초 간격)
    elif time.time() - last_keepalive > 30:
        yield ": keepalive\n\n"
        last_keepalive = time.time()

    await asyncio.sleep(0.1)
```

1.4 백엔드 테스트

1. 의존성 설치 (python-jose 필요)

cd backend
pip install python-jose[cryptography]

2. pytest 실행

PYTHONPATH=backend pytest backend/tests/ -q

기대 결과: 8 passed (sse_token 테스트 1개 추가)

체크리스트:

- [ ] sse_token.py 파일 추가 완료
- [ ] main.py 라우터 등록 완료
- [ ] transport_sse.py 수정 완료 (verify_sse_token)
- [ ] keepalive 패치 적용 완료
- [ ] pytest 8 passed 확인

---

2. Phase 2: 프론트엔드 통합 (40분)

2.1 파일 배치

useSSETokenManager 추가

cp uav_v33_final/frontend/src/utils/useSSETokenManager.ts frontend/src/utils/

Map3D 컴포넌트 추가

mkdir -p frontend/src/components/Map3D
cp uav_v33_final/frontend/src/components/Map3D/*.tsx frontend/src/components/Map3D/

App.tsx 교체

cp uav_v33_final/frontend/src/App.tsx frontend/src/

2.2 useSSE.ts 수정

useSSETokenManager 통합:

// frontend/src/hooks/useSSE.ts
import { useSSETokenManager } from "../utils/useSSETokenManager";

export function useSSE({ sessionId, token, baseUrl = "" }:
UseSSEOptions) {
// [NEW] sse_token 자동 재발급
const { sseToken, loading: tokenLoading } = useSSETokenManager({
accessToken: token, // Bearer token
baseUrl,
});

const connect = useCallback(() => {
if (!sseToken) return; // 토큰 준비 전 대기

```
// URL에 sse_token 사용
const url = `${baseUrl}/telemetry/stream?session_id=${sessionId}&sse_token=
const es = new EventSource(url);

// 이하 기존 로직...
```

}, [sessionId, sseToken, baseUrl]); // deps에 sseToken 추가

useEffect(() => {
if (sseToken) connect();
}, [sseToken, connect]);

return { telemetry, connected, qosLevel };
}

2.3 의존성 설치

cd frontend

Three.js 및 React Three Fiber

npm install three @react-three/fiber @react-three/drei

TypeScript 타입 정의

npm install -D @types/three

2.4 환경변수 설정

frontend/.env.development

VITE_API_URL=http://localhost:8000

frontend/.env.production

VITE_API_URL=https://api.uav-system.com

2.5 프론트엔드 테스트

Vitest 실행

npx vitest run src/tests/sseStore.test.ts

기대 결과: 6 passed

체크리스트:

- [ ] useSSETokenManager.ts 추가 완료
- [ ] Map3D 컴포넌트 3개 추가 완료
- [ ] App.tsx 교체 완료
- [ ] useSSE.ts 수정 완료 (sseToken 연동)
- [ ] 의존성 설치 완료 (three, @react-three/fiber)
- [ ] vitest 6 passed 확인

---

3. Phase 3: E2E 테스트 설정 (20분)

3.1 파일 배치

playwright 설정

cp uav_v33_final/playwright.config.ts ./

E2E 테스트

mkdir -p e2e
cp uav_v33_final/e2e/reconnect.spec.ts e2e/

3.2 Playwright 설치

Playwright 설치

npm install -D @playwright/test

Chromium 브라우저 설치

npx playwright install chromium

3.3 환경변수 설정

.env.e2e

E2E_BASE_URL=http://localhost:5173
E2E_API_URL=http://localhost:8000

3.4 E2E 실행

백엔드 실행 (별도 터미널)

cd backend
uvicorn app.main:app --reload --port 8000

프론트엔드 실행 (별도 터미널)

cd frontend
npm run dev

E2E 테스트 실행

npx playwright test

기대 결과: 2 passed

E2E 디버깅 모드:

UI 모드로 실행 (시각적 디버깅)

npx playwright test --ui

특정 테스트만 실행

npx playwright test reconnect.spec.ts

체크리스트:

- [ ] playwright.config.ts 추가 완료
- [ ] e2e/reconnect.spec.ts 추가 완료
- [ ] Playwright 설치 완료
- [ ] 백엔드 실행 중 (localhost:8000)
- [ ] 프론트엔드 실행 중 (localhost:5173)
- [ ] npx playwright test 2 passed 확인

---

4. Phase 4: 통합 검증 (30분)

4.1 수동 검증 절차

로그인 및 SSE 연결:

1. 브라우저: http://localhost:5173
2. 로그인: demo / demo123
3. 개발자 도구 → Network 탭 → Filter: EventStream
4. SSE 연결 확인: /telemetry/stream?session_id=...\&sse_token=...
5. HUD QoS 표시 확인: □ normal

3D View 전환 테스트:

1. 우측 상단 "3D View" 버튼 클릭
2. MapLibre terrain 3D 렌더링 확인
3. pitch 60도, 지형 고도 표현 확인
4. "2D View" 버튼 클릭 → 기존 지도 복귀

4.2 QoS 시뮬레이션

부하 테스트 실행:

백엔드 디렉토리에서

python backend/scripts/load_slow_client.py
--url "http://localhost:8000/telemetry/stream?session_id=sess_test&sse_token=TEMP_TOKEN"
--clients 5
--read-delay 0.8
--duration 30

관찰 사항:

1) HUD QoS 레벨 변화: □ normal → □ medium → □ critical
2) 3D View 모드 시: QoS critical → 자동 2D 폴백 (pitch=0, terrain off)
3) 경고 배너 표시: "□ High Load — 3D FPV disabled"
4) 부하 종료 후: QoS normal 복원 → 3D 재활성화 가능

4.3 SSE 토큰 재발급 검증

Network 탭 관찰:

1. POST /auth/sse-token 호출 확인 (최초 로그인)
2. Response: {"sse_token": "eyJ...", "expires_at": ..., "ttl_sec": 60}
3. 50초 후: POST /auth/sse-token 자동 재호출 확인
4. EventSource 재연결 확인 (새 sse_token 사용)

4.4 통합 검증 체크리스트

백엔드:

- [ ] pytest 8 passed
- [ ] POST /auth/sse-token 200 OK
- [ ] SSE 연결 성공 (EventStream)
- [ ] keepalive 30초 주기 확인 (Network 탭)
- [ ] /stats 드롭률 0% (무부하)

프론트엔드:

- [ ] vitest 6 passed
- [ ] npm run dev 빌드 성공
- [ ] SSE 연결 성공 (EventSource)
- [ ] HUD QoS 표시 □
- [ ] 3D View 전환 버튼 동작
- [ ] terrain 3D 렌더링 확인

E2E:

- [ ] npx playwright test 2 passed
- [ ] force_disconnect 재연결 성공
- [ ] SSE 토큰 자동 재발급 확인

통합:

- [ ] QoS 레벨 변화 □ → □ → □
- [ ] QoS critical → 3D off 자동
- [ ] 부하 해제 → 3D 복원
- [ ] 토큰 60초 → 재발급 → 재연결

- [ ] 5분 연속 운영 무중단

---

5. Production 배포 전략

5.1 Docker Compose 아키텍처

docker-compose.prod.yml

```
version: '3.8'

services:
backend:
build:
context: ./backend
dockerfile: Dockerfile.prod
environment:
- DATABASE_URL=postgresql://user:pass@db:5432/uav
- REDIS_URL=redis://redis:6379/0
- SECRET_KEY=${SECRET_KEY}
ports:
- "8000:8000"
depends_on:
- db
- redis
restart: unless-stopped

frontend:
build:
context: ./frontend
dockerfile: Dockerfile.prod
args:
- VITE_API_URL=https://api.uav-system.com
ports:
- "80:80"
- "443:443"
volumes:
- ./nginx/ssl:/etc/nginx/ssl:ro
- ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
depends_on:
```

```
- backend
restart: unless-stopped

db:
image: postgres:15-alpine
environment:
- POSTGRES_USER=uav_user
- POSTGRES_PASSWORD=${DB_PASSWORD}
- POSTGRES_DB=uav_db
volumes:
- postgres_data:/var/lib/postgresql/data
restart: unless-stopped

redis:
image: redis:7-alpine
command: redis-server --appendonly yes
volumes:
- redis_data:/data
restart: unless-stopped

volumes:
postgres_data:
redis_data:
```

5.2 백엔드 Dockerfile

```
# backend/Dockerfile.prod

FROM python:3.11-slim

WORKDIR /app

# 의존성 설치

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

소스 복사

COPY . .

Gunicorn으로 실행 (production ASGI server)

CMD ["gunicorn", "app.main:app",
"--workers", "4",
"--worker-class", "uvicorn.workers.UvicornWorker",
"--bind", "0.0.0.0:8000",
"--access-logfile", "-",
"--error-logfile", "-"]

5.3 프론트엔드 Dockerfile

frontend/Dockerfile.prod

FROM node:18-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
ARG VITE_API_URL
ENV VITE_API_URL=${VITE_API_URL}
RUN npm run build

Nginx 서빙

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx/nginx.conf /etc/nginx/nginx.conf
EXPOSE 80 443
CMD ["nginx", "-g", "daemon off;"]

5.4 Nginx 설정 (HTTPS/WSS)

nginx/nginx.conf

upstream backend {
server backend:8000;
}

server {
listen 80;
server_name uav-system.com www.uav-system.com;
return 301 https://$server_name$request_uri;
}

server {
listen 443 ssl http2;
server_name uav-system.com www.uav-system.com;

```
ssl_certificate /etc/nginx/ssl/fullchain.pem;
ssl_certificate_key /etc/nginx/ssl/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;

# 프론트엔드
location / {
    root /usr/share/nginx/html;
    try_files $uri $uri/ /index.html;
}

# 백엔드 API
location /api/ {
    proxy_pass http://backend/;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
    proxy_set_header X-Forwarded-Proto $scheme;

    # SSE 최적화
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 3600s;
    proxy_send_timeout 3600s;
  }

  # SSE 전용 경로
  location /telemetry/stream {
    proxy_pass http://backend/telemetry/stream;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 3600s;
    chunked_transfer_encoding off;

    # CORS (필요 시)
    add_header Access-Control-Allow-Origin *;
  }
```

}

5.5 환경변수 관리

.env.production (Git 제외)

SECRET_KEY=<64자 랜덤 문자열>
DB_PASSWORD=<강력한 비밀번호>
MAPBOX_TOKEN=<Mapbox Access Token>

.gitignore 추가:

.env.production
nginx/ssl/*.pem

5.6 SSL 인증서 발급 (Let's Encrypt)

Certbot 설치

sudo apt-get install certbot

인증서 발급

sudo certbot certonly --standalone
-d uav-system.com
-d www.uav-system.com
--email admin@uav-system.com
--agree-tos

인증서 복사

sudo cp /etc/letsencrypt/live/uav-system.com/fullchain.pem nginx/ssl/
sudo cp /etc/letsencrypt/live/uav-system.com/privkey.pem nginx/ssl/

자동 갱신 설정 (crontab)

0 0 1 * * certbot renew --quiet && docker-compose restart frontend

---

6. 배포 실행 절차

6.1 사전 준비

1. 저장소 클론

git clone https://github.com/org/uav-system.git
cd uav-system

2. 환경변수 설정

cp .env.example .env.production
nano .env.production # SECRET_KEY, DB_PASSWORD 설정

3. SSL 인증서 발급 (5.6 절차)

4. Mapbox 토큰 발급 (https://account.mapbox.com)

6.2 배포 실행

Docker Compose 빌드 및 실행

docker-compose -f docker-compose.prod.yml up -d --build

로그 확인

docker-compose logs -f backend
docker-compose logs -f frontend

헬스체크

curl https://api.uav-system.com/health
curl https://uav-system.com

6.3 배포 검증

백엔드 API 확인:

Health check

curl https://api.uav-system.com/health

기대: {"status":"ok","tick":...}

SSE token 발급

curl -X POST https://api.uav-system.com/auth/login
-H "Content-Type: application/json"
-d '{"username":"demo","password":"demo123"}'

기대: {"access_token":"..."}

curl -X POST https://api.uav-system.com/auth/sse-token
-H "Authorization: Bearer <access_token>"

기대: {"sse_token":"...","expires_at":...,"ttl_sec":60}

프론트엔드 접속:

1. 브라우저: https://uav-system.com
2. 로그인: demo / demo123
3. SSE 연결 확인 (개발자 도구)
4. 3D View 전환 확인
5. QoS 시뮬레이션 확인

---

7. 모니터링 및 로깅

7.1 애플리케이션 로그

Backend 로그 집계:

docker-compose.prod.yml

services:
backend:
logging:
driver: "json-file"
options:
max-size: "10m"
max-file: "3"

로그 확인:

실시간 로그

docker-compose logs -f backend

로그 검색

docker-compose logs backend | grep ERROR

로그 파일 직접 확인

docker exec -it <container_id> cat /app/logs/app.log

7.2 시스템 메트릭

Prometheus + Grafana 통합:

docker-compose.monitoring.yml

services:
prometheus:
image: prom/prometheus
volumes:
- ./prometheus.yml:/etc/prometheus/prometheus.yml
- prometheus_data:/prometheus
ports:
- "9090:9090"

grafana:
image: grafana/grafana
volumes:
- grafana_data:/var/lib/grafana
ports:
- "3000:3000"
environment:
- GF_SECURITY_ADMIN_PASSWORD=admin123

volumes:
prometheus_data:
grafana_data:

Prometheus 설정:

prometheus.yml

global:
scrape_interval: 15s

scrape_configs:

- job_name: 'backend'
  static_configs:
  - targets: ['backend:8000']
    metrics_path: '/metrics'

7.3 알림 설정

Slack 웹훅 통합:

backend/app/monitoring.py

import requests

def send_alert(message: str, level: str = "warning"):
webhook_url = os.getenv("SLACK_WEBHOOK_URL")
if not webhook_url:
return

```
payload = {
    "text": f"[{level.upper()}] UAV System: {message}",
    "username": "UAV Monitor",
    "icon_emoji": ":robot_face:",
}

requests.post(webhook_url, json=payload)
```

사용 예시

if drop_rate > 0.5:
send_alert(f"High drop rate: {drop_rate:.2%}", level="critical")

---

8. 백업 및 복구

8.1 데이터베이스 백업

자동 백업 스크립트

#!/bin/bash

backup.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR=/backups

docker exec uav_db pg_dump -U uav_user uav_db >
DATE.sql

7일 이상 된 백업 삭제

find $BACKUP_DIR -name "uav_db_*.sql" -mtime +7 -delete

Crontab 등록

0 2 * * * /opt/uav-system/backup.sh

8.2 복구 절차

데이터베이스 복구

docker exec -i uav_db psql -U uav_user uav_db <
/backups/uav_db_20260225_020000.sql

Redis 복구 (AOF 파일)

docker cp /backups/appendonly.aof uav_redis:/data/
docker-compose restart redis

---

9. 보안 체크리스트

9.1 인증 및 인가

- [ ] JWT SECRET_KEY 64자 이상 랜덤 문자열
- [ ] SSE 토큰 60초 TTL 적용
- [ ] Bearer token HTTPS 전송 강제
- [ ] 비밀번호 bcrypt 해싱 (saltRounds >= 12)
- [ ] CORS 설정 (허용 도메인 제한)

9.2 네트워크 보안

- [ ] HTTPS/TLS 1.2+ 강제
- [ ] HTTP → HTTPS 리다이렉트
- [ ] SSL 인증서 유효성 확인
- [ ] nginx 보안 헤더 적용:
  - X-Frame-Options: DENY
  - X-Content-Type-Options: nosniff
  - Strict-Transport-Security: max-age=31536000

9.3 애플리케이션 보안

- [ ] SQL Injection 방어 (ORM 사용)
- [ ] XSS 방어 (입력 검증, 출력 이스케이프)
- [ ] CSRF 토큰 적용 (상태 변경 API)
- [ ] Rate limiting (API 호출 제한)
- [ ] 로그에 민감정보 제외 (비밀번호, 토큰)

---

10. 트러블슈팅 가이드

10.1 SSE 연결 끊김

증상: EventSource 연결 1분 내 끊김

원인 및 해결:

1) keepalive 미전송 → backend event_gen() 확인
2) nginx 버퍼링 → proxy_buffering off 확인
3) 방화벽 타임아웃 → proxy_read_timeout 3600s 확인

10.2 3D terrain 렌더링 실패

증상: terrain 타일 404 에러

원인 및 해결:

1) Mapbox token 미설정 → TerrainView.tsx 확인
2) CORS 에러 → Access-Control-Allow-Origin 헤더 확인
3) 타일 URL 오류 → Network 탭에서 실제 요청 URL 확인

10.3 Docker 컨테이너 재시작 반복

증상: docker-compose ps 결果 Restarting 상태

원인 및 해결:

로그 확인

docker-compose logs <service_name>

일반적 원인:

1) DB 연결 실패 → DATABASE_URL 확인

2) 포트 충돌 → netstat -tulpn | grep <port>

3) 환경변수 누락 → .env.production 확인

10.4 E2E 테스트 실패

증상: Playwright test timeout

원인 및 해결:

1) 백엔드 미실행 → uvicorn 실행 확인
2) 프론트엔드 미실행 → npm run dev 확인
3) 포트 충돌 → playwright.config.ts webServer 설정 확인

---

11. 성능 최적화 가이드

11.1 백엔드 최적화

DB 쿼리 최적화:

N+1 쿼리 방지 (eager loading)

from sqlalchemy.orm import joinedload

users = session.query(User).options(
joinedload(User.sessions)
).all()

Redis 캐싱:

자주 조회되는 데이터 캐싱

import redis
cache = redis.Redis(host='redis', port=6379)

def get_session_stats(session_id: str):
cache_key = f"stats:{session_id}"

```
cached = cache.get(cache_key)
if cached:
return json.loads(cached)
```

```
stats = compute_stats(session_id)
cache.setex(cache_key, 60, json.dumps(stats))
return stats
```

11.2 프론트엔드 최적화

React.memo 적용:

```
// AdaptiveViewer.tsx
export const AdaptiveViewer = React.memo(({ telemetry }) => {
// 렌더링 로직...
}, (prev, next) => {
// telemetry가 동일하면 리렌더 방지
return prev.telemetry.timestamp === next.telemetry.timestamp;
});
```

Three.js LOD:

```
// FPVCamera.tsx
import { LOD } from 'three';

const lod = new LOD();
lod.addLevel(highDetailMesh, 0);
lod.addLevel(mediumDetailMesh, 50);
lod.addLevel(lowDetailMesh, 100);
scene.add(lod);
```

11.3 네트워크 최적화

SSE 압축 (gzip):

backend/app/transport_sse.py

```
from starlette.responses import StreamingResponse

return StreamingResponse(
event_gen(),
media_type="text/event-stream",
headers={
"Content-Encoding": "gzip", # 추가
"Cache-Control": "no-cache",
},
)
```

CDN 적용:

nginx/nginx.conf

```
location /static/ {
alias /usr/share/nginx/html/static/;
expires 1y;
add_header Cache-Control "public, immutable";
}
```

---

12. 최종 체크리스트

Phase 1-4 실행

- [ ] Phase 1: 백엔드 통합 완료 (pytest 8 passed)
- [ ] Phase 2: 프론트엔드 통합 완료 (vitest 6 passed)
- [ ] Phase 3: E2E 설정 완료 (playwright 2 passed)
- [ ] Phase 4: 통합 검증 완료 (DoD 전체 체크)

Production 배포

- [ ] Docker Compose 설정 완료
- [ ] Dockerfile 작성 완료 (backend, frontend)
- [ ] nginx 설정 완료 (HTTPS/WSS)
- [ ] SSL 인증서 발급 완료

- [ ] 환경변수 설정 완료 (.env.production)
- [ ] docker-compose up 성공
- [ ] 헬스체크 200 OK
- [ ] 프론트엔드 접속 성공 (HTTPS)
- [ ] SSE 연결 성공 (WSS)
- [ ] 3D View 전환 성공
- [ ] QoS 시뮬레이션 성공

모니터링 및 보안

- [ ] 로그 집계 설정 완료
- [ ] Prometheus + Grafana 설정 (선택)
- [ ] Slack 알림 설정 완료 (선택)
- [ ] 백업 스크립트 작성 완료
- [ ] 보안 체크리스트 전체 확인
- [ ] 트러블슈팅 가이드 숙지

---

13. 다음 회차 추천

우선순위 1: 사용자 피드백

- QoS 레벨 수동 고정 옵션 (자동 폴백 비활성화)
- FPV 녹화 기능 (MediaRecorder API)
- 텔레메트리 히스토리 재생 (타임라인)

우선순위 2: 고급 모니터링

- ELK Stack 통합 (Elasticsearch, Logstash, Kibana)
- APM 도구 통합 (New Relic, Datadog)
- 실시간 대시보드 (Grafana 고급 설정)

우선순위 3: 스케일링

- Kubernetes 마이그레이션 (Docker Swarm 대안)
- 로드 밸런서 설정 (HAProxy, nginx+)
- 데이터베이스 읽기 복제 (PostgreSQL replication)

References

[1] v3.4 4단계 통합 설계 문서 (2026-02-25). VERIFICATION_v34.md.

제작: AI Assistant (머스크 사고 + 제1원칙 적용)
버전: UAV v3.4 Production Guide
날짜: 2026-02-25 (수) 12:30 KST

UAV v3.4 Production 배포 가이드

제작일: 2026년 2월 25일 (수) 12:30 KST
기반: v3.4 4단계 통합 완성
목표: Phase 1-4 실행 → 통합 검증 → Production 배포

Executive Summary

v3.4 설계를 Production 환경으로 배포하기 위한 완전한 실행 가이드입니다. Phase 1-4 순차 실행 체크리스트, 통합 검증 절차, Docker 기반 Production 배포 전략을 포함합니다[1].

배포 목표

- Phase 1-4 순차 실행 (백엔드 → 프론트 → E2E → 검증)
- Definition of Done 전체 체크리스트 완료
- Docker Compose 기반 Production 환경 구축
- HTTPS/WSS 보안 통신 적용
- 모니터링 및 로깅 시스템 구축

---

1. Phase 1: 백엔드 통합 (30분)

1.1 파일 배치

sse_token.py 추가:

파일 위치: backend/app/sse_token.py

uav_v33_final/backend/app/sse_token.py → 프로젝트 backend/app/

cp uav_v33_final/backend/app/sse_token.py backend/app/

main.py 라우터 등록:

backend/app/main.py

from app.sse_token import router as sse_token_router

app.include_router(sse_token_router)

1.2 transport_sse.py 수정

기존 코드 수정 지점:

backend/app/transport_sse.py

from .sse_token import verify_sse_token

@router.get("/telemetry/stream")
async def telemetry_stream(
request: Request,
session_id: str = Query(...),
sse_token: str = Query(...), # token → sse_token 변경
last_event_id: str | None = Header(default=None, alias="Last-Event-ID"),
):
# sse_token 검증 (60초 TTL)
try:
user = verify_sse_token(sse_token)

except HTTPException as e:
raise e

```
# 이하 기존 로직...
```

1.3 keepalive 패치 적용

event_gen() 함수 수정:

backend/app/transport_sse.py

async def event_gen():
last_keepalive = time.time()

```
while not await request.is_disconnected():
    # 1. 텔레메트리 전송
    if queue.qsize() > 0:
        item = queue.get()
        yield f"data: {json.dumps(item)}\n\n"
        last_keepalive = time.time()

    # 2. keepalive (30초 간격)
    elif time.time() - last_keepalive > 30:
        yield ": keepalive\n\n"
        last_keepalive = time.time()

    await asyncio.sleep(0.1)
```

1.4 백엔드 테스트

1. 의존성 설치 (python-jose 필요)

cd backend
pip install python-jose[cryptography]

2. pytest 실행

PYTHONPATH=backend pytest backend/tests/ -q

기대 결과: 8 passed (sse_token 테스트 1개 추가)

체크리스트:

- [ ] sse_token.py 파일 추가 완료
- [ ] main.py 라우터 등록 완료
- [ ] transport_sse.py 수정 완료 (verify_sse_token)
- [ ] keepalive 패치 적용 완료
- [ ] pytest 8 passed 확인

---

2. Phase 2: 프론트엔드 통합 (40분)

2.1 파일 배치

useSSETokenManager 추가

cp uav_v33_final/frontend/src/utils/useSSETokenManager.ts frontend/src/utils/

Map3D 컴포넌트 추가

mkdir -p frontend/src/components/Map3D
cp uav_v33_final/frontend/src/components/Map3D/*.tsx frontend/src/components/Map3D/

App.tsx 교체

cp uav_v33_final/frontend/src/App.tsx frontend/src/

2.2 useSSE.ts 수정

useSSETokenManager 통합:

// frontend/src/hooks/useSSE.ts
import { useSSETokenManager } from "../utils/useSSETokenManager";

export function useSSE({ sessionId, token, baseUrl = "" }:
UseSSEOptions) {
// [NEW] sse_token 자동 재발급
const { sseToken, loading: tokenLoading } = useSSETokenManager({
accessToken: token, // Bearer token
baseUrl,
});

const connect = useCallback(() => {
if (!sseToken) return; // 토큰 준비 전 대기

```
// URL에 sse_token 사용
const url = `${baseUrl}/telemetry/stream?session_id=${sessionId}&sse_token=
const es = new EventSource(url);

// 이하 기존 로직...
```

}, [sessionId, sseToken, baseUrl]); // deps에 sseToken 추가

useEffect(() => {
if (sseToken) connect();
}, [sseToken, connect]);

return { telemetry, connected, qosLevel };
}

2.3 의존성 설치

cd frontend

Three.js 및 React Three Fiber

npm install three @react-three/fiber @react-three/drei

TypeScript 타입 정의

npm install -D @types/three

2.4 환경변수 설정

frontend/.env.development

VITE_API_URL=http://localhost:8000

frontend/.env.production

VITE_API_URL=https://api.uav-system.com

2.5 프론트엔드 테스트

Vitest 실행

npx vitest run src/tests/sseStore.test.ts

기대 결과: 6 passed

체크리스트:

- [ ] useSSETokenManager.ts 추가 완료
- [ ] Map3D 컴포넌트 3개 추가 완료
- [ ] App.tsx 교체 완료

- [ ] useSSE.ts 수정 완료 (sseToken 연동)
- [ ] 의존성 설치 완료 (three, @react-three/fiber)
- [ ] vitest 6 passed 확인

---

3. Phase 3: E2E 테스트 설정 (20분)

3.1 파일 배치

playwright 설정

cp uav_v33_final/playwright.config.ts ./

E2E 테스트

mkdir -p e2e
cp uav_v33_final/e2e/reconnect.spec.ts e2e/

3.2 Playwright 설치

Playwright 설치

npm install -D @playwright/test

Chromium 브라우저 설치

npx playwright install chromium

3.3 환경변수 설정

.env.e2e

E2E_BASE_URL=http://localhost:5173
E2E_API_URL=http://localhost:8000

3.4 E2E 실행

백엔드 실행 (별도 터미널)

```
cd backend
uvicorn app.main:app --reload --port 8000
```

프론트엔드 실행 (별도 터미널)

```
cd frontend
npm run dev
```

E2E 테스트 실행

```
npx playwright test
```

기대 결과: 2 passed

E2E 디버깅 모드:

UI 모드로 실행 (시각적 디버깅)

```
npx playwright test --ui
```

특정 테스트만 실행

```
npx playwright test reconnect.spec.ts
```

체크리스트:

- [ ] playwright.config.ts 추가 완료
- [ ] e2e/reconnect.spec.ts 추가 완료
- [ ] Playwright 설치 완료
- [ ] 백엔드 실행 중 (localhost:8000)
- [ ] 프론트엔드 실행 중 (localhost:5173)

- [ ] npx playwright test 2 passed 확인

---

4. Phase 4: 통합 검증 (30분)

4.1 수동 검증 절차

로그인 및 SSE 연결:

1. 브라우저: http://localhost:5173
2. 로그인: demo / demo123
3. 개발자 도구 → Network 탭 → Filter: EventStream
4. SSE 연결 확인: /telemetry/stream?session_id=...\&sse_token=...
5. HUD QoS 표시 확인: □ normal

3D View 전환 테스트:

1. 우측 상단 "3D View" 버튼 클릭
2. MapLibre terrain 3D 렌더링 확인
3. pitch 60도, 지형 고도 표현 확인
4. "2D View" 버튼 클릭 → 기존 지도 복귀

4.2 QoS 시뮬레이션

부하 테스트 실행:

백엔드 디렉토리에서

python backend/scripts/load_slow_client.py
--url "http://localhost:8000/telemetry/stream?session_id=sess_test&sse_token=TEMP_TOKEN"
--clients 5
--read-delay 0.8
--duration 30

관찰 사항:

1) HUD QoS 레벨 변화: □ normal → □ medium → □ critical
2) 3D View 모드 시: QoS critical → 자동 2D 폴백 (pitch=0, terrain off)

3) 경고 배너 표시: "⚠ High Load — 3D FPV disabled"
4) 부하 종료 후: QoS normal 복원 → 3D 재활성화 가능

4.3 SSE 토큰 재발급 검증

Network 탭 관찰:

1. POST /auth/sse-token 호출 확인 (최초 로그인)
2. Response: {"sse_token": "eyJ...", "expires_at": ..., "ttl_sec": 60}
3. 50초 후: POST /auth/sse-token 자동 재호출 확인
4. EventSource 재연결 확인 (새 sse_token 사용)

4.4 통합 검증 체크리스트

백엔드:

- [ ] pytest 8 passed
- [ ] POST /auth/sse-token 200 OK
- [ ] SSE 연결 성공 (EventStream)
- [ ] keepalive 30초 주기 확인 (Network 탭)
- [ ] /stats 드롭률 0% (무부하)

프론트엔드:

- [ ] vitest 6 passed
- [ ] npm run dev 빌드 성공
- [ ] SSE 연결 성공 (EventSource)
- [ ] HUD QoS 표시 ⚠
- [ ] 3D View 전환 버튼 동작
- [ ] terrain 3D 렌더링 확인

E2E:

- [ ] npx playwright test 2 passed
- [ ] force_disconnect 재연결 성공
- [ ] SSE 토큰 자동 재발급 확인

통합:

- [ ] QoS 레벨 변화 ⚠ → ⚠ → ⚠

- [ ] QoS critical → 3D off 자동
- [ ] 부하 해제 → 3D 복원
- [ ] 토큰 60초 → 재발급 → 재연결
- [ ] 5분 연속 운영 무중단

---

5. Production 배포 전략

5.1 Docker Compose 아키텍처

docker-compose.prod.yml

```
version: '3.8'

services:
backend:
build:
context: ./backend
dockerfile: Dockerfile.prod
environment:
- DATABASE_URL=postgresql://user:pass@db:5432/uav
- REDIS_URL=redis://redis:6379/0
- SECRET_KEY=${SECRET_KEY}
ports:
- "8000:8000"
depends_on:
- db
- redis
restart: unless-stopped

frontend:
build:
context: ./frontend
dockerfile: Dockerfile.prod
args:
- VITE_API_URL=https://api.uav-system.com
ports:
- "80:80"
- "443:443"
```

```
volumes:
- ./nginx/ssl:/etc/nginx/ssl:ro
- ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
depends_on:
- backend
restart: unless-stopped

db:
image: postgres:15-alpine
environment:
- POSTGRES_USER=uav_user
- POSTGRES_PASSWORD=${DB_PASSWORD}
- POSTGRES_DB=uav_db
volumes:
- postgres_data:/var/lib/postgresql/data
restart: unless-stopped

redis:
image: redis:7-alpine
command: redis-server --appendonly yes
volumes:
- redis_data:/data
restart: unless-stopped

volumes:
postgres_data:
redis_data:
```

5.2 백엔드 Dockerfile

backend/Dockerfile.prod

```
FROM python:3.11-slim

WORKDIR /app
```

의존성 설치

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

소스 복사

```
COPY . .
```

Gunicorn으로 실행 (production ASGI server)

```
CMD ["gunicorn", "app.main:app",
"--workers", "4",
"--worker-class", "uvicorn.workers.UvicornWorker",
"--bind", "0.0.0.0:8000",
"--access-logfile", "-",
"--error-logfile", "-"]
```

5.3 프론트엔드 Dockerfile

frontend/Dockerfile.prod

```
FROM node:18-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
ARG VITE_API_URL
ENV VITE_API_URL=${VITE_API_URL}
RUN npm run build
```

Nginx 서빙

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx/nginx.conf /etc/nginx/nginx.conf
EXPOSE 80 443
CMD ["nginx", "-g", "daemon off;"]

5.4 Nginx 설정 (HTTPS/WSS)

nginx/nginx.conf

upstream backend {
server backend:8000;
}

server {
listen 80;
server_name uav-system.com www.uav-system.com;
return 301 https://$server_name$request_uri;
}

server {
listen 443 ssl http2;
server_name uav-system.com www.uav-system.com;

```
ssl_certificate /etc/nginx/ssl/fullchain.pem;
ssl_certificate_key /etc/nginx/ssl/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;

# 프론트엔드
location / {
    root /usr/share/nginx/html;
    try_files $uri $uri/ /index.html;
}
```

```
    # 백엔드 API
    location /api/ {
        proxy_pass http://backend/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # SSE 최적화
        proxy_buffering off;
        proxy_cache off;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }

    # SSE 전용 경로
    location /telemetry/stream {
        proxy_pass http://backend/telemetry/stream;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_buffering off;
        proxy_cache off;
        proxy_read_timeout 3600s;
        chunked_transfer_encoding off;

        # CORS (필요 시)
        add_header Access-Control-Allow-Origin *;
    }
```

}

5.5 환경변수 관리

.env.production (Git 제외)

```
SECRET_KEY=<64자 랜덤 문자열>
DB_PASSWORD=<강력한 비밀번호>
MAPBOX_TOKEN=<Mapbox Access Token>
```

.gitignore 추가:

```
.env.production
nginx/ssl/*.pem
```

5.6 SSL 인증서 발급 (Let's Encrypt)

Certbot 설치

```
sudo apt-get install certbot
```

인증서 발급

```
sudo certbot certonly --standalone
-d uav-system.com
-d www.uav-system.com
--email admin@uav-system.com
--agree-tos
```

인증서 복사

```
sudo cp /etc/letsencrypt/live/uav-system.com/fullchain.pem nginx/ssl/
sudo cp /etc/letsencrypt/live/uav-system.com/privkey.pem nginx/ssl/
```

자동 갱신 설정 (crontab)

0 0 1 * * certbot renew --quiet && docker-compose restart frontend

---

6. 배포 실행 절차

6.1 사전 준비

1. 저장소 클론

git clone https://github.com/org/uav-system.git
cd uav-system

2. 환경변수 설정

cp .env.example .env.production
nano .env.production # SECRET_KEY, DB_PASSWORD 설정

3. SSL 인증서 발급 (5.6 절차)

4. Mapbox 토큰 발급 (https://account.mapbox.com)

6.2 배포 실행

Docker Compose 빌드 및 실행

docker-compose -f docker-compose.prod.yml up -d --build

로그 확인

docker-compose logs -f backend
docker-compose logs -f frontend

헬스체크

curl https://api.uav-system.com/health
curl https://uav-system.com

6.3 배포 검증

백엔드 API 확인:

Health check

curl https://api.uav-system.com/health

기대: {"status":"ok","tick":...}

SSE token 발급

curl -X POST https://api.uav-system.com/auth/login
-H "Content-Type: application/json"
-d '{"username":"demo","password":"demo123"}'

기대: {"access_token":"..."}

curl -X POST https://api.uav-system.com/auth/sse-token
-H "Authorization: Bearer <access_token>"

기대: {"sse_token":"...","expires_at":...,"ttl_sec":60}

프론트엔드 접속:

1. 브라우저: https://uav-system.com
2. 로그인: demo / demo123
3. SSE 연결 확인 (개발자 도구)
4. 3D View 전환 확인
5. QoS 시뮬레이션 확인

---

7. 모니터링 및 로깅

7.1 애플리케이션 로그

Backend 로그 집계:

docker-compose.prod.yml

services:
backend:
logging:
driver: "json-file"
options:
max-size: "10m"
max-file: "3"

로그 확인:

실시간 로그

docker-compose logs -f backend

로그 검색

docker-compose logs backend | grep ERROR

로그 파일 직접 확인

docker exec -it <container_id> cat /app/logs/app.log

7.2 시스템 메트릭

Prometheus + Grafana 통합:

docker-compose.monitoring.yml

```
services:
prometheus:
image: prom/prometheus
volumes:
- ./prometheus.yml:/etc/prometheus/prometheus.yml
- prometheus_data:/prometheus
ports:
- "9090:9090"

grafana:
image: grafana/grafana
volumes:
- grafana_data:/var/lib/grafana
ports:
- "3000:3000"
environment:
- GF_SECURITY_ADMIN_PASSWORD=admin123
```

volumes:
prometheus_data:
grafana_data:

Prometheus 설정:

prometheus.yml

global:
scrape_interval: 15s

scrape_configs:

- job_name: 'backend'
  static_configs:
  - targets: ['backend:8000']
    metrics_path: '/metrics'

7.3 알림 설정

Slack 웹훅 통합:

backend/app/monitoring.py

import requests

def send_alert(message: str, level: str = "warning"):
webhook_url = os.getenv("SLACK_WEBHOOK_URL")
if not webhook_url:
return

```
payload = {
    "text": f"[{level.upper()}] UAV System: {message}",
    "username": "UAV Monitor",
    "icon_emoji": ":robot_face:",
}

requests.post(webhook_url, json=payload)
```

사용 예시

if drop_rate > 0.5:
send_alert(f"High drop rate: {drop_rate:.2%}", level="critical")

---

8. 백업 및 복구

8.1 데이터베이스 백업

자동 백업 스크립트

#!/bin/bash

backup.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR=/backups

docker exec uav_db pg_dump -U uav_user uav_db >
DATE.sql

7일 이상 된 백업 삭제

find $BACKUP_DIR -name "uav_db_*.sql" -mtime +7 -delete

Crontab 등록

0 2 * * * /opt/uav-system/backup.sh

8.2 복구 절차

데이터베이스 복구

```
docker exec -i uav_db psql -U uav_user uav_db <
/backups/uav_db_20260225_020000.sql
```

Redis 복구 (AOF 파일)

```
docker cp /backups/appendonly.aof uav_redis:/data/
docker-compose restart redis
```

---

9. 보안 체크리스트

9.1 인증 및 인가

- [ ] JWT SECRET_KEY 64자 이상 랜덤 문자열
- [ ] SSE 토큰 60초 TTL 적용
- [ ] Bearer token HTTPS 전송 강제
- [ ] 비밀번호 bcrypt 해싱 (saltRounds >= 12)
- [ ] CORS 설정 (허용 도메인 제한)

9.2 네트워크 보안

- [ ] HTTPS/TLS 1.2+ 강제
- [ ] HTTP → HTTPS 리다이렉트
- [ ] SSL 인증서 유효성 확인
- [ ] nginx 보안 헤더 적용:
  - X-Frame-Options: DENY
  - X-Content-Type-Options: nosniff
  - Strict-Transport-Security: max-age=31536000

9.3 애플리케이션 보안

- [ ] SQL Injection 방어 (ORM 사용)
- [ ] XSS 방어 (입력 검증, 출력 이스케이프)
- [ ] CSRF 토큰 적용 (상태 변경 API)

- [ ] Rate limiting (API 호출 제한)
- [ ] 로그에 민감정보 제외 (비밀번호, 토큰)

---

10. 트러블슈팅 가이드

10.1 SSE 연결 끊김

증상: EventSource 연결 1분 내 끊김

원인 및 해결:

1) keepalive 미전송 → backend event_gen() 확인
2) nginx 버퍼링 → proxy_buffering off 확인
3) 방화벽 타임아웃 → proxy_read_timeout 3600s 확인

10.2 3D terrain 렌더링 실패

증상: terrain 타일 404 에러

원인 및 해결:

1) Mapbox token 미설정 → TerrainView.tsx 확인
2) CORS 에러 → Access-Control-Allow-Origin 헤더 확인
3) 타일 URL 오류 → Network 탭에서 실제 요청 URL 확인

10.3 Docker 컨테이너 재시작 반복

증상: docker-compose ps 결과 Restarting 상태

원인 및 해결:

로그 확인

docker-compose logs <service_name>

일반적 원인:

1) DB 연결 실패 → DATABASE_URL 확인

2) 포트 충돌 → netstat -tulpn | grep <port>

3) 환경변수 누락 → .env.production 확인

10.4 E2E 테스트 실패

증상: Playwright test timeout

원인 및 해결:

1) 백엔드 미실행 → uvicorn 실행 확인
2) 프론트엔드 미실행 → npm run dev 확인
3) 포트 충돌 → playwright.config.ts webServer 설정 확인

---

11. 성능 최적화 가이드

11.1 백엔드 최적화

DB 쿼리 최적화:

N+1 쿼리 방지 (eager loading)

from sqlalchemy.orm import joinedload

users = session.query(User).options(
joinedload(User.sessions)
).all()

Redis 캐싱:

자주 조회되는 데이터 캐싱

import redis
cache = redis.Redis(host='redis', port=6379)

def get_session_stats(session_id: str):
cache_key = f"stats:{session_id}"
cached = cache.get(cache_key)
if cached:
return json.loads(cached)

```
stats = compute_stats(session_id)
cache.setex(cache_key, 60, json.dumps(stats))
return stats
```

11.2 프론트엔드 최적화

React.memo 적용:

// AdaptiveViewer.tsx
export const AdaptiveViewer = React.memo(({ telemetry }) => {
// 렌더링 로직...
}, (prev, next) => {
// telemetry가 동일하면 리렌더 방지
return prev.telemetry.timestamp === next.telemetry.timestamp;
});

Three.js LOD:

```
// FPVCamera.tsx
import { LOD } from 'three';

const lod = new LOD();
lod.addLevel(highDetailMesh, 0);
lod.addLevel(mediumDetailMesh, 50);
lod.addLevel(lowDetailMesh, 100);
scene.add(lod);
```

11.3 네트워크 최적화

SSE 압축 (gzip):

backend/app/transport_sse.py

```
from starlette.responses import StreamingResponse

return StreamingResponse(
event_gen(),
media_type="text/event-stream",
headers={
"Content-Encoding": "gzip", # 추가
"Cache-Control": "no-cache",
},
)
```

CDN 적용:

nginx/nginx.conf

```
location /static/ {
alias /usr/share/nginx/html/static/;
expires 1y;
add_header Cache-Control "public, immutable";
}
```

---

12. 최종 체크리스트

Phase 1-4 실행

- [ ] Phase 1: 백엔드 통합 완료 (pytest 8 passed)
- [ ] Phase 2: 프론트엔드 통합 완료 (vitest 6 passed)
- [ ] Phase 3: E2E 설정 완료 (playwright 2 passed)
- [ ] Phase 4: 통합 검증 완료 (DoD 전체 체크)

Production 배포

- [ ] Docker Compose 설정 완료
- [ ] Dockerfile 작성 완료 (backend, frontend)
- [ ] nginx 설정 완료 (HTTPS/WSS)
- [ ] SSL 인증서 발급 완료
- [ ] 환경변수 설정 완료 (.env.production)
- [ ] docker-compose up 성공
- [ ] 헬스체크 200 OK
- [ ] 프론트엔드 접속 성공 (HTTPS)
- [ ] SSE 연결 성공 (WSS)
- [ ] 3D View 전환 성공
- [ ] QoS 시뮬레이션 성공

모니터링 및 보안

- [ ] 로그 집계 설정 완료
- [ ] Prometheus + Grafana 설정 (선택)
- [ ] Slack 알림 설정 완료 (선택)
- [ ] 백업 스크립트 작성 완료
- [ ] 보안 체크리스트 전체 확인
- [ ] 트러블슈팅 가이드 숙지

---

13. 다음 회차 추천

우선순위 1: 사용자 피드백

- QoS 레벨 수동 고정 옵션 (자동 폴백 비활성화)
- FPV 녹화 기능 (MediaRecorder API)
- 텔레메트리 히스토리 재생 (타임라인)

우선순위 2: 고급 모니터링

- ELK Stack 통합 (Elasticsearch, Logstash, Kibana)
- APM 도구 통합 (New Relic, Datadog)
- 실시간 대시보드 (Grafana 고급 설정)

우선순위 3: 스케일링

- Kubernetes 마이그레이션 (Docker Swarm 대안)
- 로드 밸런서 설정 (HAProxy, nginx+)
- 데이터베이스 읽기 복제 (PostgreSQL replication)

References

[1] v3.4 4단계 통합 설계 문서 (2026-02-25). VERIFICATION_v34.md.

제작: AI Assistant (머스크 사고 + 제1원칙 적용)
버전: UAV v3.4 Production Guide
날짜: 2026-02-25 (수) 12:30 KST